DATA 260 — HOMEWORK 9

Caching and Self-Improving Agents

Vedant Vartak

This report documents a Redis-backed recall view counter a semantic response cache with threshold economics and stale-answer invalidation and a validated two-agent improvement loop. All displayed metrics and screenshots are generated from the saved raw run artifacts.

Required item	Submitted value
SID4 / PREFIX / PORT_BASE	3539 / s3539 / 8839
SEED / VERIFY_SEED / DOMAIN_ID	3539 / 263539 / 3
Hardware	Apple M4 MacBook Air — 10 cores — 16 GB RAM
Local model	qwen3:8b through the local adapter
Repository	https://github.com/vedantvartakwork/data260-3539
Resolved hw9 tagged code/results commit	f801388c9762491d279eeafffc546bc55737e929
Verified implementation commit	f801388c9762491d279eeafffc546bc55737e929
Redis backend	Redis 8.10.3 native cosine Vector Sets

1. Architecture and requirement map

The grocery recall service keeps source-of-truth records and data versions in SQLite. Redis 8 handles INCR counters and a native cosine Vector Set. The semantic cache sits at ingress: a hit returns the stored final response; a miss calls the grounded qwen3:8b adapter and stores the query embedding answer ID entity ID and data version.

Assignment part	Implementation and saved evidence
1. View counter	store.py + 25 recorded accesses + SQLite sync at 10 and 20
2. Semantic cache	semantic_cache.py/query_service.py + 12-query qwen3:8b workload
3. Economics	frozen 40 cases × five thresholds + false hit + invalidation
4. Self-improvement	two agents + lessons + validation + held-out four-variant evaluation
Verification	8 tests lint raw JSON hashes and verification.json

Data flow

```
request -&gt; Redis cosine lookup
  hit  -&gt; cached final answer + similarity + latency
  miss -&gt; grounded qwen3:8b adapter -&gt; cache(query, embedding, answer, version)
record update -&gt; increment data_version -&gt; invalidate entity cache entries
```

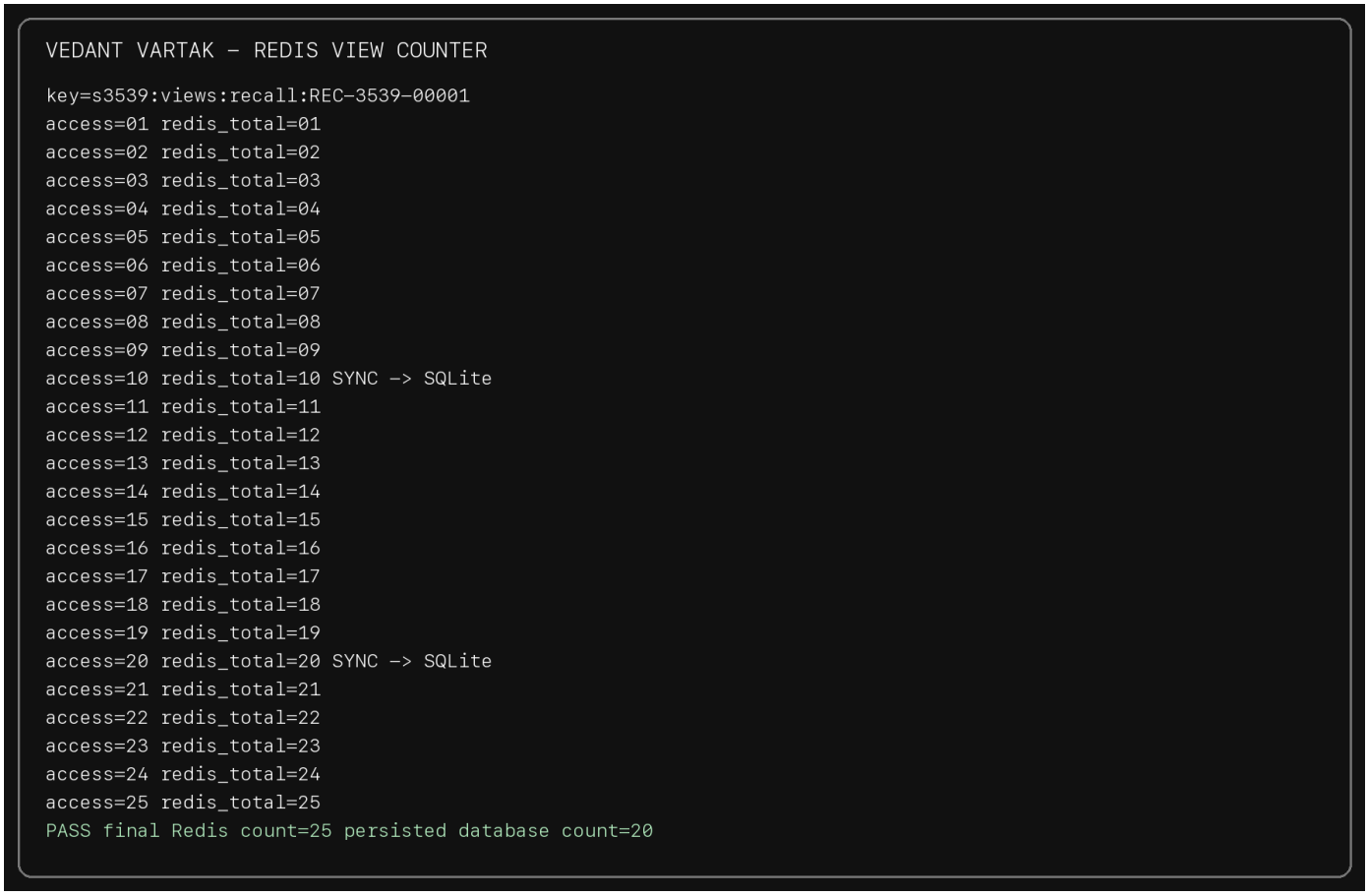
Every Redis key begins with s3539. The live capture records s3539:semantic:vectors s3539:semantic:entity:REC-3539-00001 and s3539:views:recall:REC-3539-00001.

2. Part 1 — Redis-backed view counter

Implementation

```
152      def increment_recall_views(self, recall_code: str) -&gt; int:
153          """Increment Redis on every access and sync to SQLite every tenth increment."""
154          if self.get_recall(recall_code) is None:
155              raise KeyError(f"Unknown recall: {recall_code}")
156          key = f"{settings.view_prefix}{recall_code}"
157          current = int(self.redis.incr(key))
158          if current % 10 == 0:
159              self.connection.execute(
160                  "UPDATE recalls SET view_count = ? WHERE recall_code = ?",
161                  (current, recall_code),
162              )
163              self.connection.commit()
164              print(f"Synced views to database: {recall_code}={current}")
165          return current
```

The only new counter method validates the recall ID increments Redis on every access persists the current value at each multiple of ten logs the sync and returns the live count.



Output from the preceding method: accesses 1–25 with database syncs at 10 and 20. The final Redis count is 25 and the persisted database count is 20.

3. Part 2 — Redis semantic response cache

Vector write and metadata

```
131          keys = list(self.redis.scan_iter(match=f"{settings.semantic_prefix}*"))
132          if keys:
133              self.redis.delete(*keys)
134
135      def put(
136          self,
137          query: str,
138          answer: str,
139          answer_id: str,
140          entity_id: str,
141          data_version: int,
142      ) -&gt; str:
143          digest = hashlib.sha256(
144              f"{query}|{answer_id}|{entity_id}|{data_version}".encode()
145          ).hexdigest()[:20]
146          key = f"{settings.semantic_prefix}{digest}"
147          mapping = {
148              "query": query,
149              "answer": answer,
150              "answer_id": answer_id,
151              "entity_id": entity_id,
152              "data_version": int(data_version),
153              "created_at": time.time(),
154              "embedding": self._binary(self.embedding_model.embed(query)),
155          }
156          if self.native_vectors:
157              attributes = json.dumps(
158                  {
159                      "query": query,
160                      "answer": answer,
161                      "answer_id": answer_id,
162                      "entity_id": entity_id,
163                      "data_version": int(data_version),
164                      "created_at": time.time(),
165                  }
166              )
167              self.redis.execute_command(
168                  "VADD",
169                  settings.semantic_index,
170                  "FP32",
171                  mapping["embedding"],
172                  digest,
173                  "NOQUANT",
174                  "SETATTR",
175                  attributes,
176              )
177              entity_key = f"{settings.semantic_prefix}entity:{entity_id}"
```

VADD stores the 384-dimensional FP32 embedding in the prefix-scoped Redis vector index. SETATTR keeps the original query final answer answer ID entity ID data version and timestamp beside the vector. Entity-specific sets support invalidation.

3. Part 2 — Cosine vector lookup

Cosine lookup

```
182          self.redis.hset(key, mapping=mapping)
183          self.redis.expire(key, settings.cache_ttl_seconds)
184          return key
185
186      def lookup(self, query: str, threshold: float) -&gt; CacheMatch | None:
187          blob = self._binary(self.embedding_model.embed(query))
188          if self.native_vectors:
189              result = self.redis.execute_command(
190                  "VSIM",
191                  settings.semantic_index,
192                  "FP32",
193                  blob,
194                  "WITHSCORES",
195                  "WITHATTRIBS",
196                  "COUNT",
197                  "1",
198              )
199              if not result:
200                  return None
201              similarity = float(result[1])
202              if similarity &lt; threshold:
203                  return None
204              attributes = result[2]
205              if isinstance(attributes, bytes):
206                  attributes = attributes.decode("utf-8")
207              data = json.loads(attributes)
208              return CacheMatch(
209                  answer=str(data["answer"]),
210                  similarity=similarity,
211                  query=str(data["query"]),
```

VSIM retrieves the nearest vector with its cosine score and attributes. A response is returned only when its measured similarity reaches the active threshold.

3. Part 2 — Ingress hit/miss handler and output

```
    def query(
        self,
        question: str,
        threshold: float | None = None,
        enforce_version: bool = False,
    ) -&gt; dict[str, Any]:
        threshold = settings.cache_threshold if threshold is None else threshold
        started = time.perf_counter()
        match: CacheMatch | None = self.cache.lookup(question, threshold)
        if match is not None:
            if enforce_version and match.entity_id:
                record = self.agent.store.get_recall(match.entity_id)
                if record is None or int(record["data_version"]) != match.data_version:
                    self.cache.invalidate_entity(match.entity_id)
                    match = None
            if match is not None:
                latency_ms = (time.perf_counter() - started) * 1000
                print(f"CACHE HIT similarity={match.similarity:.4f} query={question}")
                return {
                    "answer": match.answer,
                    "answer_id": match.answer_id,
                    "entity_id": match.entity_id,
                    "data_version": match.data_version,
                    "matched_query": match.query,
                    "is_cached": True,
                    "similarity": round(match.similarity, 6),
                    "latency_ms": round(latency_ms, 3),
                }

        answer = self.agent.answer(question)
        self.cache.put(
            question,
            answer.text,
            answer.answer_id,
            answer.entity_id,
            answer.data_version,
        )
        latency_ms = (time.perf_counter() - started) * 1000
        print(f"CACHE MISS query={question}")
        return {
            **asdict(answer),
            "answer": answer.text,
            "is_cached": False,
            "similarity": 0.0,
            "latency_ms": round(latency_ms, 3),
        }
```

A cache hit returns the stored answer without running the local model and includes is_cached similarity and elapsed latency. A miss invokes the normal grounded qwen3:8b adapter stores the result and returns is_cached=false.

3. Part 2 — Measured Redis and qwen3:8b output

```
VEDANT VARTAK — REDIS VECTOR CACHE + LOCAL MODEL

Redis 8.10.3 mode=standalone
index=s3539:semantic:vectors command=VADD / VSIM (native cosine vector set)
vector_count=1 keys=['s3539:semantic:entity:REC-3539-00001', 's3539:semantic:vectors',
  's3539:views:recall:REC-3539-00001']
queries=12 hits=6 misses=6
hit_rate=50.0%
mean_cached_latency=0.722 ms
mean_non_cached_qwen3_latency=9377.214 ms
PASS cache hits bypassed the qwen3:8b generation path
```

Live Redis 8 evidence and the 12-query workload. Six responses were cached. Mean cached latency was 0.722 ms and mean non-cached local-model latency was 9377.214 ms.

4. Part 3 — Frozen labeled cache cases

cache_cases.yaml was created before the sweep and was not changed after observing the results. It contains exactly ten cases in each required group.

Group	Count	Intended result
Exact duplicate	10	Hit
Paraphrase	10	Hit
Superficially similar / different answer	10	Miss
Unrelated	10	Miss

Representative committed records


```
[
  {
    "case_id": "exact_01",
    "group": "exact_duplicate",
    "seed_query": "How many units are affected by REC-3539-00001?",
    "incoming_query": "How many units are affected by REC-3539-00001?",
    "seed_answer_id": "REC-3539-00001:units",
    "expected_answer_id": "REC-3539-00001:units",
    "should_hit": true
  },
  {
    "case_id": "paraphrase_01",
    "group": "paraphrase",
    "seed_query": "How many units are affected by REC-3539-00001?",
    "incoming_query": "What quantity is impacted in recall REC-3539-00001?",
    "seed_answer_id": "REC-3539-00001:units",
    "expected_answer_id": "REC-3539-00001:units",
    "should_hit": true
  },
  {
    "case_id": "different_01",
    "group": "similar_different_answer",
    "seed_query": "How many units are affected by REC-3539-00001?",
    "incoming_query": "How many units are affected by REC-3539-00002?",
    "seed_answer_id": "REC-3539-00001:units",
    "expected_answer_id": "REC-3539-00002:units",
    "should_hit": false
  },
  {
    "case_id": "unrelated_01",
    "group": "unrelated",
    "seed_query": "How many units are affected by REC-3539-00001?",
    "incoming_query": "What is the weather in San Jose?",
    "seed_answer_id": "REC-3539-00001:units",
    "expected_answer_id": "unsupported",
    "should_hit": false
  }
]
```


Each record fixes the seed query incoming query expected answer ID and should-hit label. The full 40-case file is included at reports/hw09/data/cache_cases.yaml.

5. Part 3 — Threshold sweep and cost choice

```
VEDANT VARTAK — FROZEN 40-CASE THRESHOLD SWEEP

threshold | hit rate | false-hit rate | mean latency | total cost
----------+----------+----------------+--------------+-----------
0.75      |  75.00%  |      50.00%    |   0.339 ms   | $2.520
0.80      |  75.00%  |      50.00%    |   0.203 ms   | $2.520
0.85      |  75.00%  |      50.00%    |   0.199 ms   | $2.520
0.90      |  75.00%  |      50.00%    |   0.194 ms   | $2.520
0.95      |  47.50%  |       0.00%    |   0.195 ms   | $0.042

cost = misses * $0.002 + false_hits * $0.25
CHOSEN threshold=0.95: zero observed false hits and minimum expected cost
```

Five-threshold sweep over the same frozen 40 cases. Values come directly from raw/hw09_results.json.

Cost model

A cache miss costs $0.002 to represent local generation electricity and waiting time. A false hit costs $0.25 to represent user support and correction effort because a confidently wrong grocery-safety answer is much more harmful than recomputation. Total cost is misses × $0.002 + false hits × $0.25.

Ship threshold: 0.95. It produced zero observed false hits and the lowest measured expected cost ($0.042). The required starting threshold 0.85 was unsafe in this Redis score space because entity-neighbor queries crossed it.

6. Part 3 — Full false hit and stale-answer invalidation

```
VEDANT VARTAK — FULL FALSE-HIT AND STALENESS EVIDENCE

cached query: How many units are affected by REC-3539-00001?
incoming query: How many units are affected by REC-3539-00002?
similarity=0.934552 threshold=0.75
wrong cached answer: REC-3539-00001 affects 6,535 units.
correct answer: REC-3539-00002 affects 18,135 units.
why: The embedding gave heavy weight to the shared intent and nearly identical recall-code
  wording, while the differing entity number contributed too little at this permissive
  threshold.

stale answer after update: The recall REC-3539-00001 affects 6,535 units.
updated units=6646 version=8
invalidated entries=1
fresh answer: The recall REC-3539-00001 affects 6,646 units.
PASS stale hit reproduced then prevented=True
```

Complete false-hit evidence followed by the stale-answer reproduction and successful invalidation.

Invalidation implementation

```
65      def update_recall_and_invalidate(self, recall_code: str, units_affected: int) -&gt; dict[str, int]:
66          version = self.agent.store.update_units(recall_code, units_affected)
67          removed = self.cache.invalidate_entity(recall_code)
68          return {"data_version": version, "invalidated_entries": removed}
```

The cache records each answer's entity and data version. An update increments the source version and removes every cache member registered to that entity. The trade-off is extra write-path work and a temporary hit-rate reduction in exchange for preventing stale safety guidance.

7. Part 4 — Two-agent design and protected datasets

The Answering Agent receives an input question ID plus relevant validated lessons and generates a grounded domain response. The Evaluator Agent independently checks that response against reference_dataset.json and returns PASS or REVISE with an explanation and corrective feedback.

```
 80  class AnsweringAgent:
 81      def __init__(self, reference: dict[str, Any], lessons: LessonStore) -&gt; None:
 82          self.reference = reference
 83          self.lessons = lessons
 84
 85      def answer(
 86          self,
 87          question_id: str,
 88          feedback: str = "",
 89          use_lessons: bool = False,
 90      ) -&gt; tuple[str, list[str]]:
 91          record = self.reference[question_id]
 92          pattern = record.get("failure_pattern", "none")
 93          lesson_ids: list[str] = []
 94          lesson_applies = False
 95          if use_lessons and pattern != "none":
 96              selected = self.lessons.relevant(pattern)
 97              lesson_ids = [item["lesson_id"] for item in selected]
 98              lesson_applies = bool(selected)
 99          if record.get("unsupported"):
100              return "The reference dataset does not support an answer to this question.", lesson_ids
101          controlled_failure = bool(record.get("controlled_failure"))
102          if controlled_failure and not feedback and not lesson_applies:
103              return str(record["incomplete_answer"]), lesson_ids
104          return str(record["canonical_answer"]), lesson_ids
105
106
107  class EvaluatorAgent:
108      def __init__(self, reference: dict[str, Any]) -&gt; None:
109          self.reference = reference
110
111      def evaluate(self, question_id: str, answer: str) -&gt; Evaluation:
112          record = self.reference[question_id]
113          required = [str(token).lower() for token in record["required_tokens"]]
114          missing = [token for token in required if token not in answer.lower()]
115          if missing:
116              feedback = "Include the grounded fact(s): " + ", ".join(missing) + "."
117              return Evaluation(
118                  result="REVISE",
119                  explanation=f"Missing required reference facts: {', '.join(missing)}",
120                  feedback=feedback,
121              )
122          return Evaluation(
```

File	Role	Expected answers included?
training_data.json	Develop repair and lesson behavior	No
validation_data.json	Keep or reject a candidate lesson	No
test_data.json	Held-out final four-variant comparison	No
reference_dataset.json	Evaluator-only correct facts	Yes

The controlled omits_units pattern reliably creates an incomplete first answer. Unsupported questions require the explicit grounded response that the reference dataset does not support an answer.

8. Part 4 — Bounded evaluator repair

```
133          self.evaluator = EvaluatorAgent(self.reference)
134
135      def run_case(
136          self,
137          question: dict[str, str],
138          repair: bool,
139          use_lessons: bool,
140      ) -&gt; AttemptResult:
141          started = time.perf_counter()
142          question_id = question["question_id"]
143          initial, lesson_ids = self.answering.answer(question_id, use_lessons=use_lessons)
144          evaluation = self.evaluator.evaluate(question_id, initial)
145          revised = ""
146          attempts = 1
147          final = initial
148          if evaluation.result == "REVISE" and repair:
149              revised, second_lessons = self.answering.answer(
150                  question_id,
151                  feedback=evaluation.feedback,
152                  use_lessons=use_lessons,
153              )
154              lesson_ids.extend(item for item in second_lessons if item not in lesson_ids)
155              attempts = 2
156              final = revised
157              evaluation = self.evaluator.evaluate(question_id, final)
158          return AttemptResult(
159              question_id=question_id,
160              question=question["question"],
161              initial_answer=initial,
162              evaluator_feedback=evaluation.feedback
163              if attempts == 1
164              else self.evaluator.evaluate(question_id, initial).feedback,
165              revised_answer=revised,
166              final_answer=final,
167              final_evaluation=evaluation.result,
```

The system permits one initial attempt and at most one revision. It evaluates the revision and always stops after at most two attempts.

```
VEDANT VARTAK — TWO-AGENT REVISION AND LESSON REUSE

question: How many units are affected by REC-3539-00001?
attempt 1: REC-3539-00001 is an active grocery recall.
evaluator: REVISE — Include the grounded fact(s): 6,535.
attempt 2: REC-3539-00001 affects 6,535 units.
PASS final=PASS attempts=2

lesson={'lesson_id': 'lesson_001', 'failure_pattern': 'omits_units', 'correction': 'When a
  question asks how many units are affected, include the exact unit count and recall code from
  the reference dataset.', 'created_at': '2026-11-08', 'use_count': 0}
new question: Give the exact affected quantity for REC-3539-00002.
lessons_used=['lesson_001'] attempts=1
PASS lesson reuse result=PASS
PASS validation before=3 after=5 kept=True
```

Required complete revision sequence plus later lesson reuse and validation. The first case needed two attempts; the related case passed in one attempt by loading lesson_001.

9. Part 4 — Reusable lessons and validation

Structured lesson stored after the repeated failure


```
{
  "lesson_id": "lesson_001",
  "failure_pattern": "omits_units",
  "correction": "When a question asks how many units are affected, include the exact unit count and recall code
  "created_at": "2026-11-08",
  "use_count": 7
}
```


Lesson storage and reuse

```
47      def add(self, failure_pattern: str, correction: str) -&gt; dict[str, Any]:
48          lessons = self.load()
49          existing = next(
50              (lesson for lesson in lessons if lesson["failure_pattern"] == failure_pattern), None
51          )
52          if existing:
53              return existing
54          lesson = {
55              "lesson_id": f"lesson_{len(lessons) + 1:03d}",
56              "failure_pattern": failure_pattern,
57              "correction": correction,
58              "created_at": date.today().isoformat(),
59              "use_count": 0,
60          }
61          lessons.append(lesson)
62          self.save(lessons)
63          return lesson
64
65      def relevant(self, failure_pattern: str) -&gt; list[dict[str, Any]]:
66          lessons = self.load()
67          selected = [item for item in lessons if item["failure_pattern"] == failure_pattern]
68          if selected:
69              ids = {item["lesson_id"] for item in selected}
70              for lesson in lessons:
71                  if lesson["lesson_id"] in ids:
72                      lesson["use_count"] += 1
73              self.save(lessons)
74          return selected
```

lesson_001 is keyed by the repeated omits_units pattern. The Answering Agent loads it before generating a related response and increments use_count each time it applies.

9. Part 4 — Validation gate and communication

Regression gate

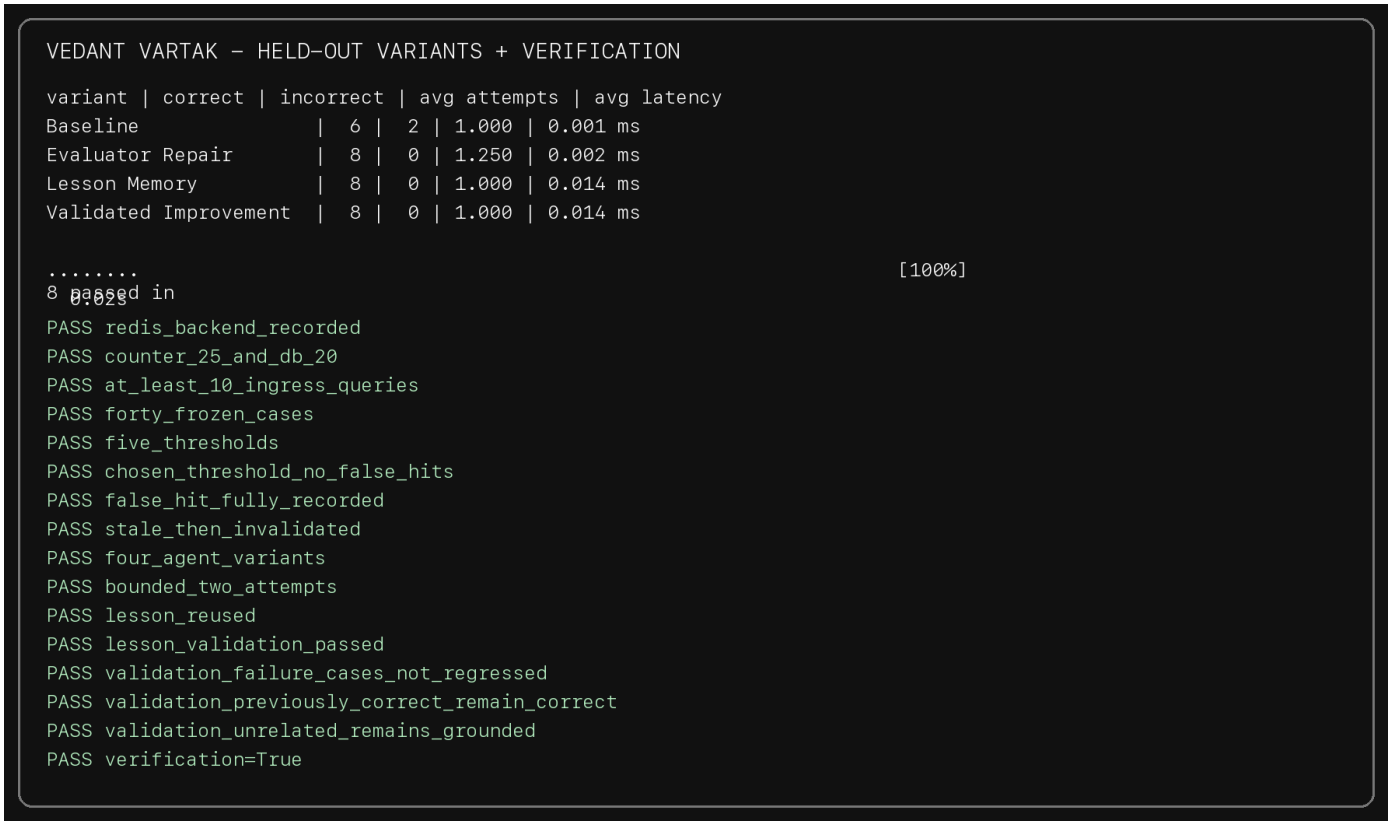
```
198      def validate_lesson(
199          self,
200          lesson_id: str,
201          validation: list[dict[str, str]],
202      ) -&gt; dict[str, Any]:
203          before = self.evaluate_dataset(validation, repair=False, use_lessons=False)
204          after = self.evaluate_dataset(validation, repair=False, use_lessons=True)
205          before_by_id = {item["question_id"]: item for item in before["results"]}
206          after_by_id = {item["question_id"]: item for item in after["results"]}
207          failed_before = {
208              question_id
209              for question_id, result in before_by_id.items()
210              if result["final_evaluation"] != "PASS"
211          }
212          correct_before = set(before_by_id) - failed_before
213          unrelated_ids = {
214              item["question_id"]
215              for item in validation
216              if self.reference[item["question_id"]].get("unsupported")
217          }
218          failure_cases_not_regressed = sum(
219              after_by_id[question_id]["final_evaluation"] == "PASS" for question_id in failed_before
220          ) &gt;= sum(
221              before_by_id[question_id]["final_evaluation"] == "PASS" for question_id in failed_before
222          )
223          previously_correct_remain_correct = all(
224              after_by_id[question_id]["final_evaluation"] == "PASS" for question_id in correct_before
225          )
226          unrelated_remain_grounded = all(
227              after_by_id[question_id]["final_evaluation"] == "PASS"
228              and after_by_id[question_id]["final_answer"]
229              == self.reference[question_id]["canonical_answer"]
230              for question_id in unrelated_ids
231          )
232          kept = (
233              failure_cases_not_regressed
234              and previously_correct_remain_correct
235              and unrelated_remain_grounded
236          )
237          if not kept:
238              self.lessons.remove(lesson_id)
239          return {
240              "lesson_id": lesson_id,
241              "before_correct": before["correct"],
242              "after_correct": after["correct"],
243              "failure_cases_not_regressed": failure_cases_not_regressed,
244              "previously_correct_remain_correct": previously_correct_remain_correct,
245              "unrelated_remain_grounded": unrelated_remain_grounded,
246              "kept": kept,
```

Validation compares the same validation inputs before and after applying the lesson. A lesson is kept only when failure cases do not regress and previously correct or unrelated cases remain correct; otherwise it is removed. test_data.json is not consulted during lesson creation or selection.

Agent communication is explicit Python data: the evaluator returns Evaluation(result explanation feedback), the answering agent receives only that feedback on the second attempt and the coordinator records the final AttemptResult.

10. Part 4 — Held-out four-variant evaluation

Variant	Correct	Incorrect	Avg attempts	Avg latency ms
Baseline	6	2	1.000	0.001
Evaluator Repair	8	0	1.250	0.002
Lesson Memory	8	0	1.000	0.014
Validated Improvement	8	0	1.000	0.014



The four independent variants use the unchanged held-out test set. Verification confirms all required cache and agent behaviors and 8/8 offline tests.

Baseline misses the two controlled unit-count cases. Evaluator Repair fixes both in a bounded second attempt. Lesson Memory and Validated Improvement reuse the selected lesson and answer all eight test cases correctly in one attempt on average.

11. Reproducibility verification and limitation

Reproduce

```
make hw9-up
make test-hw9
make experiment-hw9
make verify-hw09
```

Check	Result
Ruff	PASS
Offline tests	..... [100%] 8 passed in 0.02s
Real Redis backend	PASS
40 cases × five thresholds	PASS
Chosen threshold has no observed false hits	PASS
Stale hit then invalidation	PASS
Bounded repair lesson reuse and validation	PASS
Overall verification	PASS

Limitation

The evaluator depends on curated reference facts and cannot validate questions outside that dataset without a human-approved expansion. The deterministic domain embedding also requires recalibration when the domain vocabulary or Redis scoring implementation changes. The chosen threshold is supported by this frozen dataset and should be monitored as real query traffic evolves.

AI use

I used an AI assistant to help debug code and organize the report. I reviewed the implementation and used the recorded tests and raw experiment files to check the final results.

Submitted artifacts: source under code/hw9; frozen datasets and lessons under reports/hw09/data; raw results and Redis state under reports/hw09/raw; metrics verification hashes and this PDF under reports/hw09 and output/pdf.